

AI Video Assistant with RAG

Complete Technical Architecture & Viva/Interview Explanation Manual

Repository: lzmawarakhan/AI-Video-Assistant-With-RAG | **Stack:** Streamlit, Whisper, LangChain, ChromaDB, Groq LLM

1. EXECUTIVE SUMMARY & PROJECT OVERVIEW

The **AI Video Assistant with RAG** is an end-to-end multimodal intelligence system designed to convert video/audio content (YouTube URLs or local uploads) into structured insights and a searchable knowledge base. The system integrates automated speech recognition (ASR), local vector storage, and Retrieval-Augmented Generation (RAG) to allow users to extract transcriptions, key decisions, action items, and perform semantic Q&A.

2. HIGH-LEVEL SYSTEM ARCHITECTURE

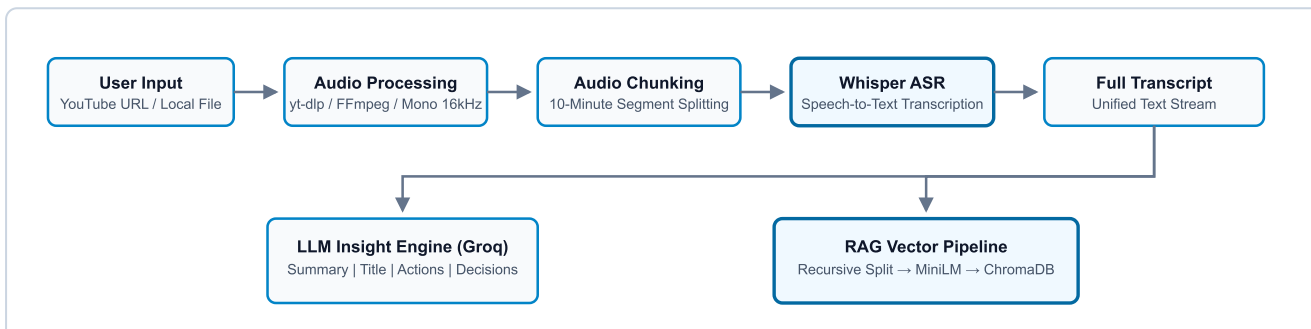
Directory Structure

```
AI-Video-Assistant-With-RAG/ ├── a
pp.py # Streamlit UI ├── main.py # Pipelin
e Orchestrator ├── requirements.txt # Proj
ect Dependencies ├── .env # Local API Keys
├── core/ | ├── transcriber.py # Whisper A
SR Engine | ├── summarized.py # LLM Summar
y Modules | ├── extractor.py # Decision/Ac
tion Engine | ├── vector_store.py # Chroma
DB & Embeddings | ├── rag_engine.py # Cont
ext Retriever + QA | ├── utils/ | ├── audio_pr
ocessor.py # Downloading & Audio Format
```

Module Responsibilities

File Component	Core Technical Responsibility
app.py	Streamlit frontend, language selection UI, session state management, chat rendering.
main.py	Central pipeline controller orchestrating ingestion, transcription, extraction, and RAG build.
audio_processor.py	Downloads via yt-dlp, standardizes audio using pydub/FFmpeg, splits into chunks.
transcriber.py	Executes local OpenAI Whisper ASR model for multilingual speech-to-text.
vector_store.py	Generates MiniLM vector embeddings and persists data into ChromaDB.
rag_engine.py	Similarity-based chunk retrieval & prompt context integration for Groq LLM.

3. MACRO DATA FLOW & PROCESSING PIPELINE



4. DEEP DIVE: INGESTION & AUDIO NORMALIZATION

To ensure high transcription accuracy and optimal memory handling across long video streams, the input audio undergoes standardized ingestion and transformation prior to model inference.

Step 1: Input Detection & Download

The system evaluates the source string in `utils/audio_processor.py`:

- Remote Links (HTTP/HTTPS):** Processed using `yt-dlp` configured with `"format": "bestaudio/best"` to extract the highest quality audio track.
- Local Uploads:** Ingested directly from temporary files (supporting `.mp4`, `.mp3`, `.wav`, `.m4a`).

Step 2: Signal Standardisation

Raw audio streams are converted using `pydub` and `FFmpeg` into a unified baseline format required by OpenAI Whisper:

- Channel Conversion:** Downmixed to 1-channel (Mono).
- Sampling Rate:** Resampled to 16,000 Hz (16 kHz).
- Format:** Encoded as uncompressed PCM 16-bit WAV.

5. AUDIO CHUNKING & WHISPER SPEECH RECOGNITION

Processing long-duration audio files directly through deep learning models can lead to memory exhaustion (OOM) and latency bottlenecks. The system mitigates this via fixed time-window chunking.

Audio Chunking Strategy: The system splits long WAV files into **10-minute distinct chunks** (e.g., `video.wav_chunk_0.wav`, `video.wav_chunk_1.wav`). Each chunk is independently fed to the Whisper ASR model, and partial transcripts are sequentially merged.

Whisper ASR Integration

Implemented in `core/transcriber.py` using local OpenAI Whisper inference:

- Model Size:** Default set to `small` via environment variable `WHISPER_MODEL` for balanced execution speed and word error rate (WER).
- Execution Environment:** Runs fully locally (CPU/GPU) without relying on external per-minute cloud API calls.
- Language Mapping:** Accepts user-selected language codes (e.g., English = `en`, Urdu = `ur`) to enforce accurate language token decoding during transcription.

LLM Insight Extraction

Once the consolidated transcript is constructed, it is routed to the Groq LLM (`openai/gpt-oss-20b`) via LangChain to extract key operational insights:

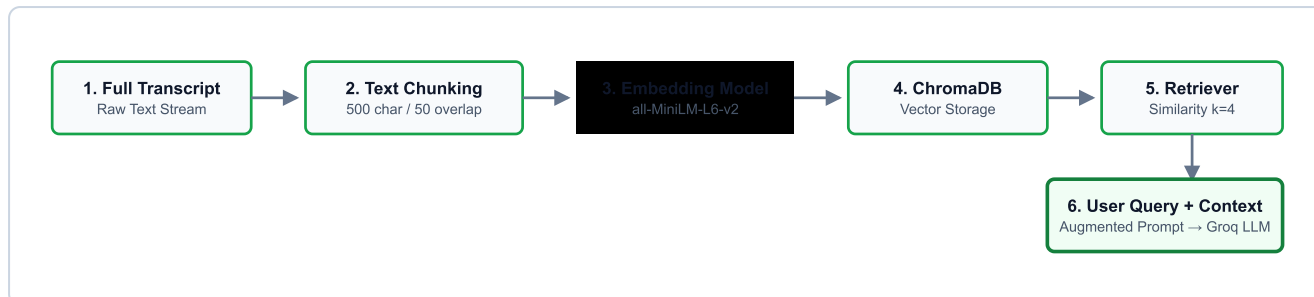
- Title Generation:** Concise topic-summarized headline.
- Executive Summary:** High-level overview of core ideas.
- Action Items:** Actionable tasks identified in discussions.
- Key Decisions:** Explicit choices or agreements made.
- Open Questions:** Unresolved topics or pending queries.

6. ANALYTICAL COMPARISON: SUMMARY VS. RAG PIPELINE

Metric / Feature	Global Summary Pipeline	RAG Q&A Pipeline
Scope	Entire video transcript sent simultaneously.	Top-\$\$\$ relevant transcript chunks retrieved per query.
Primary Objective	Broad, high-level abstraction & structured extraction.	Pinpoint answer generation grounded in specific contexts.
Tokens Consumed	Proportional to full length of transcript.	Fixed length (4 chunks × 500 characters + user query).
Context Window Risk	Can overflow context limits on long lectures/meetings.	Eliminates context limit issues via semantic filtering.
Database Dependency	None (Direct LLM Prompting).	Requires Vector DB (ChromaDB) + Embedding Model

7. DEEP DIVE: RAG ARCHITECTURE & VECTOR INDEXING

Retrieval-Augmented Generation grounds the system's responses directly in the transcript content, preventing hallucination and enabling fast querying across long media files.



Text Chunking Strategy

Executed using `RecursiveCharacterTextSplitter` in `core/vector_store.py`:

- `chunk_size = 500` : Keeps context cohesive within semantic boundaries.
- `chunk_overlap = 50` : Preserves sentence continuity across adjacent boundaries.
- **Metadata**: Every chunk is stored as a `LangChain Document` object with index metadata.

Embedding & Vector Database

High-dimensional semantic representation:

- **Embedding Model**: `all-MiniLM-L6-v2` via Hugging Face / Sentence Transformers. Maps textual semantics into dense 384-dimensional vector space.
- **Vector Store**: ChromaDB saved locally under `vector_db/` with collection name `meeting_transcript`.

8. QUERY RETRIEVAL & ANSWER GENERATION

Vector Similarity Search

1. When the user submits a question in Streamlit, the query is passed to `ask_question()` in `rag_engine.py`.
2. The question is encoded into a 384-dimensional vector using the same `all-MiniLM-L6-v2` model.
3. ChromaDB executes a cosine similarity search, returning the **top \$k=4\$ most semantically relevant chunks**.

Context Augmentation & Generation

1. The `$k=4$` retrieved chunks are concatenated to form the *Context Block*.
2. An augmented prompt combines System Instructions, Context Block, and User Query.
3. The prompt is dispatched to **Groq LLM** for fast token generation, ensuring facts are strictly grounded in video content.

```
# Conceptual RAG Prompt Structure inside rag_engine.py
SYSTEM_PROMPT = """You are an assistant for answering questions based on video transcripts. Use the following pieces of retrieved context to answer the question. If you don't know the answer, say that you don't know based on the video context."""
CONTEXT: {retrieved_chunks_1_to_4}
USER QUESTION: {user_query}
ANSWER:
```

9. TECHNICAL VIVA & INTERVIEW Q&A GUIDE

Comprehensive answers to anticipated technical questions during project evaluations and technical interviews.

Q1: Can you explain your project in one concise elevator pitch?

"My AI Video Assistant takes a YouTube URL or local media file, extracts and normalizes the audio, transcribes it using a local Whisper model, and generates AI-powered summaries, decisions, and action items using a Groq LLM. It then builds a RAG pipeline by embedding transcript chunks with all-MiniLM-L6-v2 and storing them in ChromaDB. When a user asks a question, the system performs a similarity search, retrieves relevant chunks, and passes them as context to the LLM for grounded answer generation."

Q2: Why use local Whisper instead of cloud speech APIs?

Answer: Using local Whisper eliminates recurring per-minute transcription API costs, ensures data privacy by keeping raw audio processing local, and provides fine-grained control over language parameters and model sizing without external rate limits.

Q3: Why chunk the transcript with an overlap?

Answer: `chunk_size=500` ensures text blocks fit within embedding optimal bounds. `chunk_overlap=50` prevents loss of context or split phrases at the border between adjacent chunks during similarity retrieval.

Q4: Why choose Embeddings over keyword search?

Answer: Keyword search relies on exact lexical matches and fails with synonyms. Embeddings capture *semantic meaning* in vector space, matching queries like "How does it fetch outside data?" with text containing "Retrieval accesses external context."

Q5: Why select Groq LLM for generation?

Answer: Groq provides near-instant inference processing using specialized LPUs (Language Processing Units). This dramatically reduces latency when synthesizing summaries and generating RAG chat responses.

10. PRODUCTION DEPLOYMENT & CLOUD CONSIDERATIONS

Deployment Platform

Hosted on Streamlit Community Cloud linked to GitHub repository `Izmawarakhan/AI-Video-Assistant-With-RAG`.

API Security

`.env` is excluded via `.gitignore`. Production keys are injected through Streamlit Secrets Management (`st.secrets`).

Compatibility Patch

Includes `audioop-lts` dependency in `requirements.txt` to ensure compatibility with Python 3.13+ environments where legacy `audioop` was deprecated.

Core Takeaway Flow to Remember:

Input Video → Audio Normalization (16kHz Mono) → Audio Chunking → Whisper ASR → Transcript → [1. LLM Insights / 2. Chunking + Embeddings → ChromaDB Vector DB → Retriever → Context + Query → Groq LLM → Grounded Response]